

FinFlow AI — Entry-Level Resume Guide (< 1 Yr Exp)

Realistic, Non-Senior Tone for Junior SWE (SDE-1) & Junior AI Builder

1. Tone & Style Philosophy for SDE < 1 Year Experience

Overly senior jargon like '*Architected enterprise microservices*' or '*Designed system topologies*' raises red flags for entry-level applicants. For candidate SDEs with < 1 year of experience, resume points focus on standard, believable development tasks: **Building endpoints, writing agent workflows, setting up caches, creating UI components, and containerizing services.**

2. Software Engineer LaTeX Block (Entry-Level / SDE-1)

```
{\textbf{FinFlow AI} \hfill
{\small \textit{Python \textbar{}} FastAPI \textbar{}} Next.js \textbar{}} PostgreSQL \textbar{}} Redis \textbar{}} Docker}}
{\textbar{}}\highlightlink{https://github.com/amitk2003/Multimodal-Agentic-ai}{Code} \textbar{}} \highlightlink{https://multi
\begin{itemize}
\item Built RESTful and WebSocket API endpoints to process financial data across 8 portfolio companies (\$615M+ revenue), \
\item Implemented real-time data caching and pub/sub messaging to pass execution updates, achieving \textbf{sub-second} late
\item Developed a responsive dashboard UI with interactive charts and error-tracking components to monitor \textbf{10} autom
\item Created automated PDF report generation scripts and containerized application services using \textbf{Docker} for smoo
\end{itemize}}
```

3. AI Native Builder LaTeX Block (Entry-Level / Junior AI Builder)

```
{\textbf{FinFlow AI}} \hfill
{\small \textit{Python \textbar{}} LangChain \textbar{}} Multi-LLM \textbar{}} Redis \textbar{}} FastAPI \textbar{}} Docker}}
{\textbar{}}\highlightlink{https://github.com/amitk2003/Multimodal-Agentic-ai}{Code} \textbar{}} \highlightlink{https://multi
\begin{itemize}
\item Developed a multi-agent system with 10 specialized agents to handle trial balance checks, revenue recognition rules,
\item Implemented an automated API fallback handler across multiple LLM providers to handle rate limits and \textbf{prevent}
\item Built custom evaluation prompts and agents to detect debit/credit mismatches, stale accruals, and transaction errors,
\item Integrated a shared memory layer using Redis to pass context between agents, \textbf{cutting} redundant token consumpt
\end{itemize}}
```

4. Entry-Level Interview Strategy

Interview Question	Entry-Level Authentic Answer
What was your role in this project?	I built the full-stack system from scratch using FastAPI and Next.js. I wrote the agent handlers in LangChain, connected Redis for pub/sub messaging, and created the PDF generator.
How did you handle LLM API rate limits?	I wrote a try-except fallback function in Python that checks if Gemini or Claude returns a rate limit error, and automatically switches the call to Groq or OpenAI.
How did you test your multi-agent flow?	I seeded sample portfolio financial data into PostgreSQL via FastAPI endpoints and triggered agent runs via WebSockets to test execution output in the dashboard.